Modelling Room Acoustics with a GPU

Jacob Josiah Webber

18 August 2017

MSc in High Performance Computing

The University of Edinburgh

Year of Presentation: 2017

Abstract

I made a program it took ages

Contents

List of Tables

List of Figures

List of Algorithms

Acknowledgements

I would like to extend my sincere thanks to Paul Graham and Dr Brian Hamilton for agreeing to supervise this project and all their help along the way.

Chapter 1

Introduction

"There is no competition of sounds between a nightingale and a violin."
Dejan Stojanovic

The way we perceive sound depends on the environment in which we hear it as well as how the sound was created. Listening to a sound in a space like a church or a cathedral is very different to listening in a bedroom. Being able to predict how a given environment will sound has practical use in the design of new buildings, particularly those where acoustic performance is critical. A great deal of effort goes into the design of new concert halls. This could be greatly enhanced by accurate acoustic models of spaces. The aim then, of research in this area, is to provide an accurate prediction of how a room will sound after being given its physical properties.

The key technique currently used for these simulations is the finite-difference time-domain (FDTD) method. This is used to find approximate solutions to the wave equation across a three dimensional space. The FDTD process requires that a stencil operation be performed. This stencil process must be iterated across a vast number of points in memory. The requirement for high memory bandwidth means that the problem is well suited to the use of graphics processing units (GPUs). These have high band width graphics memory and can operate on very many points at once in a massively parallel fashion.

In the work leading up to this report, some currently state of the art GPU acoustics techniques as outlined in [1] and [2] were recreated. In addition to this, a novel technique was devised and implemented that performs these simulations. The proposed approach is designed to make best use of graphics memory when rooms are irregularly shaped. This is important because graphics memory is relatively scarce.

GPUs threads can be controlled in three dimensional blocks. On NVIDIA devices these blocks can contain up to 1024 threads. When performing FDTD stencils of irregularly shaped rooms, current methods will result in there being many blocks assigned to regions that are completely outside the room being simulated. The new method proposed here decomposes rooms into blocks in space that correspond to blocks of threads. This means that space and compute is not wasted on empty blocks. These blocks are large enough that they remove some of the typical drawbacks of unstructured data approaches, but small enough that they avoid storing a significant amount of empty space. The fact that data is organised similarly to computation results in less branching logic. Branching can cause performance issues on GPUs.

This report includes some background on the theory of acoustics and difference schemes, a brief background in GPUs and an overview of the new techniques that were devised and trialled in the work leading up to this report. This is followed by description of the details of the implementation in CUDA C as well as software development, profiling and debugging strategies and tools. There is then a comparative analysis of existing and novel techniques both in terms of performance in compute time and memory use. The report finishes with some conclusions and ideas for further work.

Chapter 2

Background and Literature Review

"He who loves practice without theory is like the sailor who boards ship without a rudder and compass and never knows where he may cast."

Leonardo da Vinci

2.1 Room Acoustics

2.1.1 Classical Acoustics

Sound is modelled using the wave equation:

$$\frac{\partial^2 P}{\partial x^2} - \frac{1}{c^2}\frac{\partial^2 P}{\partial t^2} = 0 \tag{2.1}$$

where scalar field P is the deviation from the ambient air pressure, and c is the speed of sound. This result is derived by, among others, Richard Feynman in [3]. It is trivial to generalise this for 3 dimensional systems, which yields

$$\nabla^2 P - \frac{1}{c^2}\frac{\partial^2 P}{\partial t^2} = 0 \tag{2.2}$$

where

$$\nabla \equiv \hat{\mathbf{i}}\frac{\partial}{\partial x} + \hat{\mathbf{j}}\frac{\partial}{\partial y} + \hat{\mathbf{k}}\frac{\partial}{\partial z} \tag{2.3}$$

Intuitively we know that a room's acoustic properties are dependent on its size and what materials form the walls and other surfaces. This is reflected in the solutions to the wave equation, where almost all environmental data is encoded in boundary conditions imposed on the P scalar field. In real world applications these boundary conditions are too sophisticated for the wave equation to be solved analytically. This sophistication stems from both the shape of the room and the reflective properties of its boundaries. These reflective properties are a function of the frequency of the sound reflected.

2.1.2 Finite Difference Schemes

Finite difference schemes are therefore used to solve the wave equation for complex room shapes. These can provide an approximate numerical solution to the continuous wave equation.

A simple equivalence between first and second order derivatives and their corresponding central difference schemes is shown in 2.4 and 2.5.

$$\frac{df(u)}{du} \approx \frac{f(u+h) - f(u-h)}{2h} \tag{2.4}$$

$$\frac{d^2 f(u)}{du^2} \approx \frac{f(u+h) - 2f(u) + f(u-h)}{h^2} \tag{2.5}$$

This can be easily discretised if h is set to be the distance in space or time between samples.

When discretising the space within the room, the scalar field of pressure within room is represented as a three dimensional array of floating point numbers. The letters l, m, and p are chosen to represent the samples index in each spacial dimension within the array, with n giving the time step. This means $P^n_{l,m,p}$ gives an approximation to P at the point (x, y, z) at time t where $x = lH$, $y = mH$, $z = pH$ and $t = nT$. T and H are the sample rates in time and space respectively.

When equation 2.2 is expanded out, it is shown that in order to compute a numerical approximation of the wave equation, discrete equivalents are required to the derivatives in time, and in each of the three spacial dimensions.

$$\frac{\partial^2 P}{\partial t^2} = c^2 \left(\frac{\partial^2 P}{\partial x^2} + \frac{\partial^2 P}{\partial y^2} + \frac{\partial^2 P}{\partial z^2} \right) \tag{2.6}$$

To replace these continuous partial derivatives a new finite difference operator, δ, is introduced. From 2.5 the following relations can be derived by simply replacing h with H:

$$\delta_t^2 P^n_{l,m,p} \equiv \frac{1}{T^2} \left(P^{n+1}_{l,m,p} - 2P^n_{l,m,p} + P^{n-1}_{l,m,p} \right) \approx \frac{\partial^2 P}{\partial t^2} \tag{2.7}$$

$$\delta_x^2 P^n_{l,m,p} \equiv \frac{1}{H^2} \left(P^n_{l+1,m,p} - 2P^n_{l,m,p} + P^n_{l-1,m,p} \right) \approx \frac{\partial^2 P}{\partial x^2} \tag{2.8}$$

$$\delta_y^2 P_{l,m,p}^n \equiv \frac{1}{H^2}\Big(P_{l,m+1,p}^n - 2P_{l,m,p}^n + P_{l,m-1,p}^n\Big) \approx \frac{\partial^2 P}{\partial y^2} \tag{2.9}$$

$$\delta_z^2 P_{l,m,p}^n \equiv \frac{1}{H^2}\Big(P_{l,m,p+1}^n - 2P_{l,m,p}^n + P_{l,m,p-1}^n\Big) \approx \frac{\partial^2 P}{\partial z^2} \tag{2.10}$$

It is now possible to use these operators to discretise 2.6.

$$\delta_t^2 P_{l,m,p}^n = c^2\big(\delta_x^2 P_{l,m,p}^n + \delta_y^2 P_{l,m,p}^n + \delta_z^2 P_{l,m,p}^n\big) \tag{2.11}$$

Equation 2.11 can then be expanded using the results of equations 2.7-2.10. This yields

$$\begin{aligned} P_{l,m,p}^{n+1} - 2P_{l,m,p}^n + P_{l,m,p}^{n-1} = \frac{c^2T^2}{H^2}\big(&P_{l+1,m,p}^n + P_{l-1,m,p}^n + P_{l,m+1,p}^n + P_{l,m-1,p}^n \\ &+ P_{l,m,p+1}^n + P_{l,m,p-1}^n - 6P_{l,m,p}^n\big) \end{aligned} \tag{2.12}$$

Letting the Courant number be defined as

$$\lambda \equiv \frac{cT}{H} \tag{2.13}$$

and our stencil

$$S \equiv P_{l+1,m,p}^n + P_{l-1,m,p}^n + P_{l,m+1,p}^n + P_{l,m-1,p}^n + P_{l,m,p+1}^n + P_{l,m,p-1}^n \tag{2.14}$$

gives a somewhat neater version of equation 2.12

$$P_{l,m,p}^{n+1} = (2 - 6\lambda^2)P_{l,m,p}^n + \lambda^2 S - P_{l,m,p}^{n-1} \tag{2.15}$$

More than simply being neat, this is now a representation of how to calculate new values for the three dimensional array from the previous ones using a simple stencil operation. This yields a new propagation relation.

2.1.3 Boundary Conditions

All of the physical properties of the room are encoded in the boundary conditions imposed on the differential equation. Modelling different boundary materials such as room walls is the subject of significant research at present. However, the main purpose of this report is to look at the computational aspects of applying stencil operations. As such a relatively simple boundary scheme is applied.

For the purposes of a Von Neumann stencil most grid points that are inside the room will have 6 neighbours. However, on the edge of the room, there will be points that have fewer.

The amount of these neighbours is stored in a new K grid, with one value for every point. This allows for the first time rooms of irregular shape to be considered, as those points outside the room are encoded with a zero K value.

The boundary scheme used modifies equation 2.15 by replacing the 6 value with a $K_{l,m,p}$ value. All values of $P^n_{l,m,p}$ where $K_{l,m,p} = 0$ are also assumed to be zero.

$$P^{n+1}_{l,m,p} = (2 - \lambda^2 K_{l,m,p}) P^n_{l,m,p} + \lambda^2 S - P^{n-1}_{l,m,p} \tag{2.16}$$

2.2 Stencilling Operations

WHAT IS A STENCIL

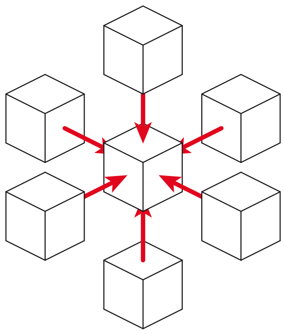

Figure 2.1: Diagram showing 3 dimensional Von Neumann stencil.
Credit: Wikimedia Commons.

Stencilling operations are in their nature memory bound problems. This is because they involve a lot of data accesses per, relatively simple, calculation. This means that GPUs, with their high bandwidth memory, have been shown to be ideally suited to perform these operations [12]. However, while GPUs have the best performance, they also have limited amounts of memory.

2.3 Graphics Processing Units (GPUs)

GPUs were originally designed for 3D game rendering [4]. For this purpose they are equipped with the ability to process large arrays of floating point numbers very efficiently, both in terms of operations per second, and per Watt. It so happens that these qualities make these devices very well suited to the field of High Performance Computing (HPC). This coincidence is a happy one indeed, as it allows the very high research and development costs of bringing a new processor to market to be spread across the much larger gaming industry. The CEO of NVIDIA, which is the largest supplier of discrete graphics cards, described the budget of several billion US dollars for the design of the latest Pascal GPU architecture as so large it would probably be enough to send someone to Mars [5].

GPUs have a number of technical features which make them particularly suited to running FDTD simulations. GPUs were shown in [1] to perform up to 2 orders of magnitude better than CPUs when running room acoustics simulations.

2.3.1 GPUs and parallelism

GPUs primarily achieve their massive performance increases over CPUs by harnessing operating in a massively parallel fashion. Two principles of parallel computing are used to make this happen. These are the *single instruction, multiple threads* (SIMT) and *simultaneous multi-threading* (SMT) approaches.

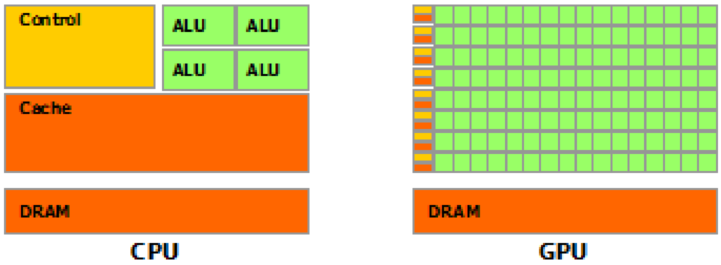

Figure 2.2: GPU and CPU architectures [6]

Single Instruction, Multiple Thread

SIMT computer instructions execute on multiple objects in memory at once. In doing so they reduce the amount of silicone that is required to perform calculations. This is because multiple operations can share so called *control units* (CUs). These determine which instructions should occur, and when branches should be taken. GPUs are organised into *streaming multiprocessors* (SMXs). These contain multiple cores and therefore *floating point units* (FPUs) but share control circuitry. This means a much higher proportion of the silicone is taken up with performing floating point operations than with a conventional CPU, where there is typically one CU per processing core. Figure 2.2 goes some way to showing the make-up of GPUs in relation to CPUs, without showing a detailed view of each multiprocessor.

Figure 2.3: The SMX architecture as in the K20 GPU [7]

More detail on the SMX is given in figure 2.3. This shows each SMX contains 192 single-precision CUDA cores, 64 double-precision units, 32 special function units (SFU), and 32 load/store units (LD/ST) [7]. Instructions are scheduled and issued in groups of 32 parallel threads called *warps*. Each SMX can run a maximum of 4 warps concurrently. This depends on there not being a *stuctural hazard*. This is when a part of the processor's hardware is needed by two or more instructions at the same time. For example, the fact that there are only 64 double precision FPUs per SMX means that not all $4 \cdot 32$ threads from the four warps can conduct floating point calculations simultaneously.

Simultaneous Multi-Threading

SMT refers to the process of running more logical threads than cores exist in hardware. This means that processes can avoid using up resources while they are idle. In GPUs this happens at the warp level. When a SMX changes which warp is active this is known as *context switching*. GPUs have special hardware adaptations that make them very efficient at doing this. They have a very large amount of registers compared with conventional

CPUs. This means that data from all *active* warps - meaning those that have been allocated to an SMX - are kept in registers. This differs from a conventional CPU where this data may have to be written out to at least L1 cache memory. The speed with which GPUs can context switch means that SMT techniques become appropriate to hide shorter latencies such as memory accesses, unlike CPUs, which are more likely to make use of SMT for threads that are idle while waiting for disk or network latencies.

As the efficiency of SMT on the GPU is dependent on large registers, performance can be limited when warps are required to complete tasks that require large amounts of registers. For ideal performance, GPUs need to be carefully programmed to ensure that they make maximal use of SMT. This reduces the amount of time hardware spends idling, and therefore increases throughput.

2.3.2 GPU Memory Hierarchies

Memory Type	Scope	Latency (approx. clock cycles)
Register File	CUDA Thread	0
Shared Memory/L1 Cache	CUDA Block / N/A	48
L2 Cache	N/A	120
Global Memory	CUDA Grid	440

Table 2.1: Memory hierarchy on NVIDIA GPU.

GPUs are tuned for throughput, rather than latency. This colours their approach to data storage as well as processing.

The previous section discusses the importance of registers for SMT. These form the top tier of the GPUs memory hierarchy. Modern NVIDIA GPUs, such as the K20 used in this project, also have two additional layers of cache memory above the global memory store. The top layer of this cache is also referred to as *shared* memory. This is because data can be explicitly loaded here and will then be shared among a group of threads defined by the programmer as a *block* (see section 2.3.3 for details). L2 cache is hardware controlled and functions similarly to CPU caches. Below this there is *global* memory. This is the largest and highest latency form of data storage on the GPU. The K20 used in experiments for this report uses GDDR5 high bandwidth graphics memory for this. This differs from DDR3/4 used in conventional computing .

GDDR5 is tuned for maximal bandwidth. It does sacrifice some latency performance in exchange for this. This is achieved by loading in larger lines of data at a time. There is much excitement in GPGPU circles about the recent arrival of *high bandwidth memory 2* (HBM2). This has even higher bandwidth than GDDR5 and has even larger access widths.

Figure 2.4 shows the advance of memory bandwidth through time on both CPUs and GPUs. The NVIDIA K20 has a peak theoretical bandwidth of $208\,\mathrm{GB\,s^{-1}}$, which is less than a third of that of some of the latest HBM2 cards. It is hoped the techniques described later in this report will be well suited to these advancements.

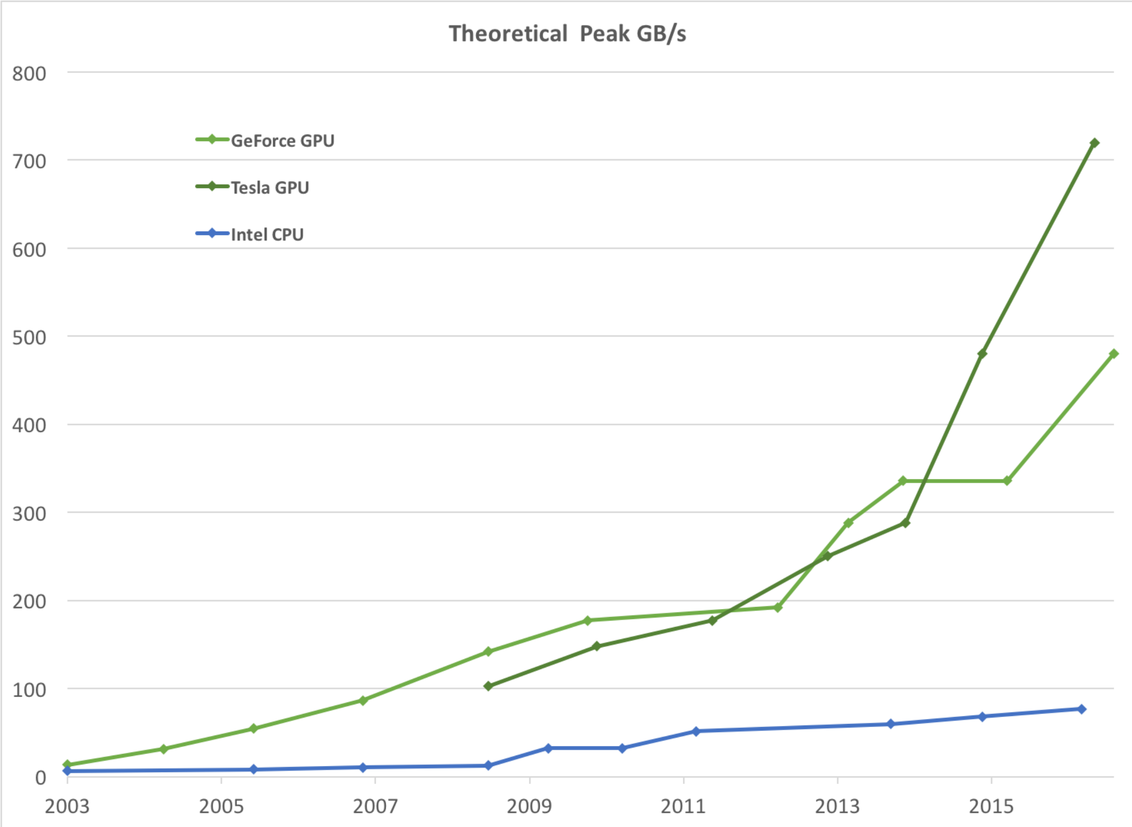

Figure 2.4: A graph from NVIDIA showing memory bandwidth of GPUs and CPUs through time [6].

2.3.3 The CUDA Paradigm

The nature of GPUs means that they need to be programmed carefully to extract performance. Computation done on the GPU is separated into kernels. In this case they were written as special CUDA C functions. These are launched in blocks, and each block can contain multiple threads. When a kernel is called, the amount of blocks and threads per block is specified.

The amount of threads per block can be specified using a `dim3` data type. This allows blocks of threads to be three dimensional. Inside a kernel each thread has an x y and z index. This can be used to for each thread to access a different data item without using control logic. For example, listing 2.1 shows a standard three dimensional array read in CUDA C - a version of CUDA that is an extension to ANSI C.

Listing 2.1: CUDA memory accesses

```
myData = data[threadIdx.z][threadIdx.y][threadIdx.x]
```

Such an access is considered idiomatic CUDA code and is well optimised in terms of memory access, particularly when the thread in the x direction corresponds to the fastest moving array index in memory. In C this is the last index specifier. Accessing data where the threads and data accesses follow the same order is known as *data coalescing*. Acessing data that is close together in memory is also important and is referred to as *data locality*.

These thread blocks are assigned to an SMX when resources become available. While they are waiting for global memory access latencies, SMT will be used to make a new warp active. As warps contain 32 threads and thread blocks can contain up to 1024, SMT can also be used within these thread blocks when multiple warps are required. The fact that threads are controlled in warps of 32 has a few ramifications for the programmer. Firstly it means that blocks that are smaller than 32 threads will waste compute capacity, as the remaining threads will execute on null data. It also means that branching logic is likely to introduce inefficiencies where it might require branching within a warp. This will result in all accessed branches being executed on all threads with those not activated operating on null data.

Shared memory has thread block scope. Once a thread is executed this memory is no longer available. Shared memory can be allocated statically or dynamically from within a kernel in CUDA C.

MORE PLEASE

2.4 The State of the Art in FDTD Acoustical Modelling

Combining the ideas as written about in sections 2.1 and 2.3, it is possible to describe the most successful techniques thus far in running FDTD room acoustics models.

2.4.1 The Bounding Box Method

SQUARE ROOM ALG

Data: Three dimensional array of $\Psi^{n}_{l,m,p}$
Result: Updated array of $\Psi^{n+1}_{l,m,p}$
Get the 3D indices of the current thread from the block and thread IDs;
if *not at the boundaries* **then**
    Compute the linear address from the 3D indices;
    Update node $\Psi^{n+1}_{l,m,p}$;
end

Algorithm 1: CUDA kernel for update on the basic 3D scheme.

FUNNY SHAPED ROOM ALG

Data: Three dimensional arrays of $\Psi^n_{l,m,p}$ and $K_{l,m,p}$
Result: Updated array of $\Psi^{n+1}_{l,m,p}$
Get the 3D indices of the current thread from the block and thread IDs;
if $K_{l,m,p}$ *is not zero* **then**
  Compute the linear address from the 3D indices;
  Update node $\Psi^{n+1}_{l,m,p}$;
end

Algorithm 2: CUDA kernel for update on an irregularly shaped room.

2.4.2 Memory Capacity

The performance of FDTD models can be measured using two factors: the computation time and the amount of memory used. Because of their high bandwidth memory and fast floating point performance, GPUs routinely perform very well in the computation time category. However, the graphics memory that they operate on is much more limited in its supply than standard DRAM. By way of example, the high end HPC targeted NVIDIA K20 device primarily used in this project has 5 GiB. This is significantly less than many modern laptops will have as main memory.

Going back to equation 2.12, it is possible to derive a stability relation for the value of the Courant number, or λ. The approximate solution to the differential equation is said to be stable if results do not grow exponentially. A full derivation is given in [10] using Von Neumann spectral analysis. This yields the important result that for stability it is required that $\lambda \leq \frac{1}{\sqrt{3}}$.

This gives a concrete relation between the sample rates in time ($\frac{1}{T}$) and space ($\frac{1}{H}$).

$$H \leq \sqrt{3}cT \tag{2.17}$$

High sample rates in time are required to avoid temporal aliasing. This is an effect where the ear lacks enough information to determine whether a signal is one of two possible values within the audible range ($\approx$ 20 Hz-20 kHz). The Nyquist limit requires that the sample rate be twice as high as the highest audible frequency to avoid this aliasing effect [11]. This requires a very large number of points for even a moderate sized room. Taking a sample rate of 44.1 kHz (as is used in audio CDs), and the speed of sound in air as $344\,\mathrm{m\,s^{-1}}$ the Courant limit becomes $0.0135\,\mathrm{m}$. This means that to model a room of even $1\,\mathrm{m}^3$ a matrix of 75 by 75 by 75 floating point numbers is required. This equates to a total array size of 421 875, or 3.22 MiB or 1.61 MiB of memory for double or single precision respectively per m^3.

As an example, The Royal Albert hall is popularly claimed to have a volume of approximately $80\,000\,\mathrm{m}^3$. This would require up to 270 GiB just to meet the Courant limit at a sample rate of 44.1 kHz.

In addition, the Courant limit simply provides a minimum number of points that will provide numerical stability. In order to get good results the number of points required is significantly higher. In the field of electro-magnetism - where FDTD techniques are most commonly used - it has been shown that many tens of samples per wavelength are required to accurately model the propagation of a wave [13]. This makes sense at a more intuitive level that the result of the Courant calculation and yields an even more taxing requirement on memory quantities.

2.4.3 Memory Bandwidth

In addition to requirements on memory capacity, the performance of FDTD simulations on given hardware is largely determined its memory bandwidth. Two factors that affect the memory bandwidth performance of FDTD simulations are the load/store efficiencies and the compute to memory access ratio (CMA).

Stencil operations by their nature have a low ratio of floating point operations (FLOPS) to memory access. For ideal performance, data will be loaded from global memory to a register or to cache and then be operated on multiple times. For the methods such as the stencil operation outlined in section 2.2, this CMA value approaches 1.0. This is because for each time step the stencil operation must be completed for all points in the grid followed by a process of synchronisation. This synchronisation overhead proves to be very expensive, as it requires that all points in utilised memory be read and written to for a relatively trivial amount of computation. Under a simple three dimensional Von Neumann stencil, as shown in figure 2.1, each point is used in 6 FLOPS. This yields a theoretical maximum CMA of 6.0. However, because of the nature of GPGPU programming, storing these values in local memory as described in section 2.3.2 results in very marginal gains, if gains at all, at the expense of more complex code.

This is because of a factor called *occupancy*, which is a measure of how many register files and how much L1 cache a compute kernel uses. High occupancy reduces the efficiency of SMT as outlined in section 2.3.1. Instead, any performance boost from this comes from hardware managed L2 cache. Cache hit rates as found in the literature are low, meaning there is a very low effective CMA for this method [1].

GPU DRAM differs from CPU DRAM primarily in that it is optimised for bandwidth rather than latency. As a result each memory load and write can operate on a relatively large chunk of memory at a time. Global memory accesses using current GDDR5 technology are optimised for $128\,\mathrm{B}$ vectorised accesses. These are by default cached in L1 and L2 which also have $128\,\mathrm{B}$ sized cache lines. These large reads can be wasted if memory accesses are scattered. This is the case when large three dimensional arrays are stored. In such an array only neighbouring points in one dimension will be neighbouring in memory and the other will have some considerable offset. This results in memory reads being wasted, cache lines being *overfetched*, or storing unnecessary data, as well as less efficient non-vectorised memory accesses being used. The first of these problems can be partially fixed by using a compilation flag to disable L1 cache. This technique is recommended in [6] for scattered memory accesses and was shown in [1] to yield marginal improvements in performance for the FDTD problem.

2.5 Other Methods

The methods outlined in this section are described in a deliberately terse way. They do not form the body of the work done for this dissertation. They are discussed here to offer some justification as to why not, as well as to give the FDTD approach some context within its field.

2.5.1 Ray Tracing

Ray tracing methods treat sound waves as rays that travel in straight lines. Sound waves are launched in multiple directions from a source position and those that cross a given output location. This has the combined effect of allowing some filtering to occur at the room boundary as well as providing a digital filtering effect from the differing path lengths of all the possible reflective paths that could lead the sound to the output location. This method yields credible results with much less computational expense than FDTD techniques and has therefore been used in the computer gaming and virtual reality industries [14]. It is, however, fundamentally limited by the fact it cannot model more complex wave effects like diffraction [2].

2.5.2 Spectral Methods

Stencil effects such as the FDTD approach can be shown to be equivalent to a convolution. It is well known that a convolution in the space domain is equivalent to a pointwise vector multiplication in the frequency domain:

$$h(x) = f \otimes g = \int_{-\infty}^{\infty} f(x-u)g(u)du = \mathcal{F}^{-1}\Big(\sqrt{2\pi}\mathcal{F}[f] \cdot \mathcal{F}[g]\Big) \tag{2.18}$$

Such an approach would remove the synchronisation overhead in the stencil operation and allow very high CMA ratios to be achieved by performing this pointwise multiplication of the Fourier transformed stencil and pressure field matrices to be performed multiple times on each memory location in turn, without having to read and write through all memory every time step.

While these approaches have been applied to stencil techniques in image processing [15] they are not applicable here. This is because the boundary conditions are the only feature that varies between different rooms and a technique for encoding these well in the frequency space is not known. In addition to model any input signal larger than one sample long it would be necessary to transform the arrays back from the frequency domain after every time step, removing any advantage from applying this method and adding a considerable overhead in the performance of two FFTs for every time step.

Chapter 3

The Proposed Data Structure and Method

"Nothing surpasses the beauty and elegance of a bad idea."

Craig Bruce

The bulk of the work in this project went into the design and implementation of a new algorithm for the computation of FDTD simulations. This section details the design of this algorithm and some of its theoretical performance benefits.

The purpose of this technique improve memory usage for irregularly shaped rooms and memory locality in large 3D FDTD acoustics simulations. The proposed method is to replace the large 3D arrays currently used with an array of structs. Each struct will store a smaller sub-array containing points in the simulation, as well as information on the location of neighbouring points to those points of the edge of each sub-array. This will allow blocks where no points are inside the room being simulated to not have to be stored in expensive GPU memory. This technique also improves data locality. The smaller 3D sub-arrays will result in neighbouring points being scattered less far apart in memory.

There is some analogue that can be found here with the field of sparse matrices. Indeed, the stencil operation can be thought of as a sparse matrix multiplication. When matrices are sparse, so-called *unstructured* approaches are used to store the data. With a standard matrix, a particular matrix index's location in memory can be derived easily. However, such an approach can require the storage of a lot of unnecessary data. With unstructured approaches, only non zero matrix elements need be stored. However, this means it is no longer possible to easily derive the memory location of an element in memory - meaning that this data must also be stored. This means that matrices must typically be very sparse indeed before unstructured approaches become practical.

If a bounding box representation can be thought of as a structured approach, and typical sparse representations as unstructured, here a technique is presented which can be thought of, and from here will be described as a *semi-structured* approach. This is where smaller blocks of structured data are stored only if they contain at least one point that is non-zero - or inside the room. These blocks are sized so that they are large enough to dwarf any additional locational data requirements, but small enough so that not too much empty space need be stored.

The nature of the CUDA paradigm can also be taken advantage. The blocks can be designed so that they correspond to a CUDA block of threads. This means that one kernel can operate on one block in a parallel fashion. This relation means that each kernel can load one contiguous block of data from global memory. This makes this approach highly suited to devices that are optimised for memory bandwidth, and therefore have larger memory transaction sizes.

3.1 The Data Structure

Listing 3.1: The block Data Struct

```
typedef real bl_array[Bz][By][Bx];
struct block {
    int up;
    int down;
    int left;
    int right;
    int fore;
    int aft;
    bl_array u;
    bl_array u1;
    char k[Bz][By][Bx];
};
```

Listing 3.1 shows the block struct. The K array is a store of the amount of internal neighbours a point has. It is set to zero if the point in question is external. A block should therefore only be stored in the array of structs if it has at least one point with a K value above zero. This means rooms whose bounding box would contain large amounts of empty space can be stored much more efficiently.

The fact that it is a stencil operation means that each kernel need not know the location of the block it is operating on - only the location of the blocks neighbours. These are therefore stored in the struct as six integers, each of which gives the block index of a neighbouring block in each direction.

These six numbers are an overhead of this method that could lead to it being less efficient in memory than the bounding box technique. However, the size of these in memory compared to the rest of the data is small.

Taking, as an example, the dimensions of the sub-arrays to be 8 by 8 by 8, then the total size requirement of a struct would be $1.002\,757$ times that of the representation without these neighbour integers stored when the user defined datatype, `real`, is set to be a double precision floating point number.

The worst case memory performance can be calculated for single precision as:

$$\frac{S_{ss}}{S_s} = 1 + \frac{24}{9 \cdot \text{Bx} \cdot \text{By} \cdot \text{Bz}} \tag{3.1}$$

and for double precision as:

$$\frac{S_{ss}}{S_s} = 1 + \frac{24}{17 \cdot \text{Bx} \cdot \text{By} \cdot \text{Bz}} \tag{3.2}$$

Where S_{ss} and S_s are the sizes of the semi-structured and structured representations in memory respectively.

3.2 The Algorithm

The new method requires a new algorithm for converting a room's representation to a semi-structured one, as well as for performing the stencil on the new data structure.

```
Data: Three dimensional array of K_{l,m,p} for a given room
Result: Array of blocks (AOB), block locations
Divide K_{l,m,p} into Bx by By by Bz sized blocks;
while not at end of blocks do
    do nothing;
    if block contains non-zero K_{l,m,p} value then
        increment T_i;
        store block's location;
    else
        store zero as block location
    end
    go to next block;
end
allocate memory for T_i block structs;
while not at end of block locations do
    if block location has non-zero value then
        copy K_{l,m,p} values to correct block in AOB;
    end
    go to next block location;
end
```

Algorithm 3: Convert bounding box to array of blocks

A sketch of this is given in algorithm 3. It requires two passes over the data as it necessary to know the amount of blocks that have internal points before memory can be allocated and the K sub-arrays can be copied to the new representation.

MORE PLEASE

3.3 Theoretical Benefits

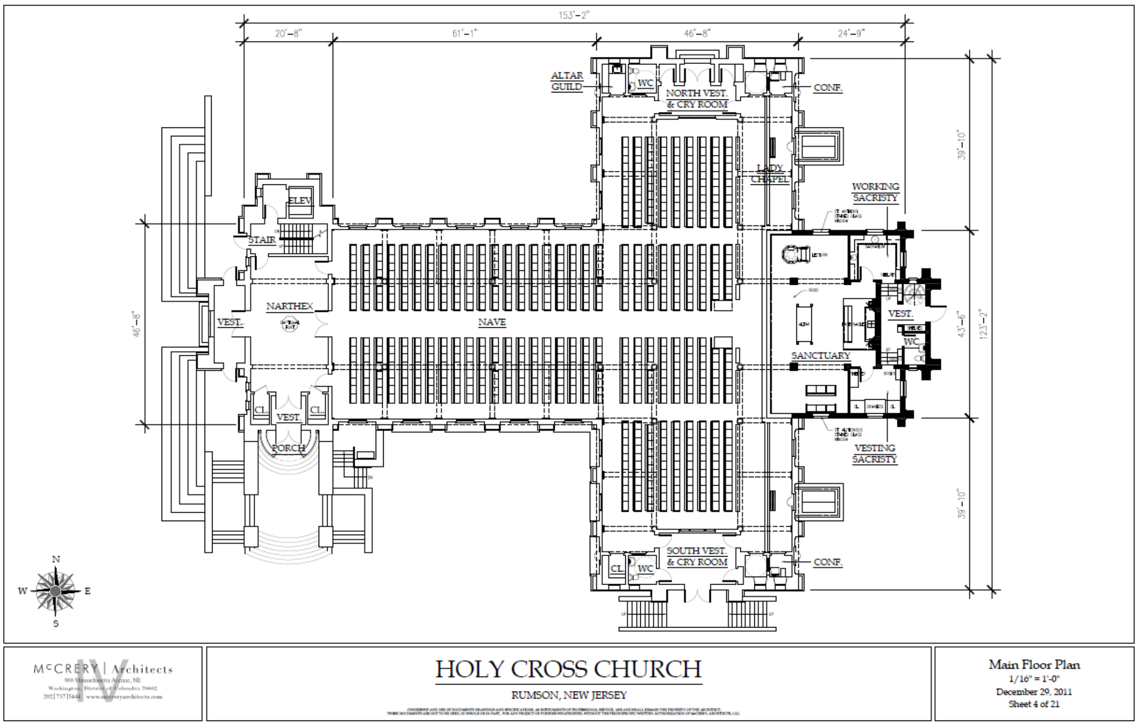

Figure 3.1: A diagram showing a schematic of a church building.
Credit: McCrery Architects.

MORE PLEASE

Chapter 4

Design and Implementation

> *"The first 90 percent of the code accounts for the first 90 percent of the development time...The remaining 10 percent of the code accounts for the other 90 percent of the development time."*
>
> Tom Cargill

The software was written in CUDA C. This is a proprietary language developed by NVIDIA and is an extension to ANSI C that allows for GPGPU programming. The CUDA C language was selected because the University of Edinburgh Acoustics and Audio Group (AAG) has an NVIDIA K20 GPU server. CUDA is a well supported way to program these devices, although the resulting programs will only run on NVIDIA hardware. Had OpenCL been used, the code would have also been able to run on graphics cards produced by AMD, who are NVIDIA's main competitor [16]. This paradigm is generally considered more complex than CUDA C, and, as the AAG does not own any AMD hardware, was not pursued. Some discussion of how future work might involve OpenCL is given in section 6.1.

The work stems from a Matlab script that implements the scheme using a bounding box as described in 2.4. There were also CUDA kernels from [1] that served as inspiration, but these did not model cases where non-cuboid rooms were used. This meant that they were of limited use. As no code exists that implements the semi-structured approach proposed here, the project was therefore largely one of software development from scratch.

4.1 Design Overview

The code was designed to be as extensible and reusable as possible. As such each experiment is divided into a separate directory. Each of these contains a CUDA C source file that performs the required experiment. These then call functions from a library that do all the tasks associated with the proposed method. This library is a selection of source files that are in a `lib` directory. They contain functions for forming the data structure as described in chapter 3, as well as generating room shapes, and performing the FDTD stencil.

MORE PLEASE

4.2 Software Development Process

4.3 Debugging and Profiling Tools

NVIDIA includes developer tools with its CUDA software. As well as the compiler, NVCC, there is a debugger, an IDE and some profiling tools.

For debugging, the `cuda-gdb` program functions like gdb, but extended to work on GPU code. The `cuda-memcheck` tool checks for memory errors such as segmentation faults. It will provide the line of code where the memory error is when the the binary is compiled using debugging flags.

For profiling there is the `nvprof` tool for the command line. There is also the NVIDIA *Visual Profiler*, which provides a GUI. This can provide details of the amount of time spent on each part of the computation, the time taken for each kernel, as well as other details such as the theoretical and achieved occupancy (see section 2.3.1). Figure FIGUREFIGURE provides a screenshot of this tool in use.

The `nsight` IDE, based on eclipse, provides access to all of these tools.

It was unfortunately not possible to use the GUI based tools when working on the K20 GPU server that was used to gather results. This is because the server is not configured to run graphical programs. It was possible to run these tools on a home desktop with an NVIDIA GTX 970 graphics card installed. This was useful for debugging is this device is capable of running CUDA code. However, it was of limited use for profiling, as the card achieved very different performance to the K20. The GTX 970 is a commodity card that lacks the double precision floating point units of the K20. It also has fewer register files and less L1 cache.

Chapter 5

Results and Analysis

5.1 Memory Usage

Very high memory usage is currently the biggest inhibiting factor to the performance of FDTD simulations.

5.1.1 Sphere

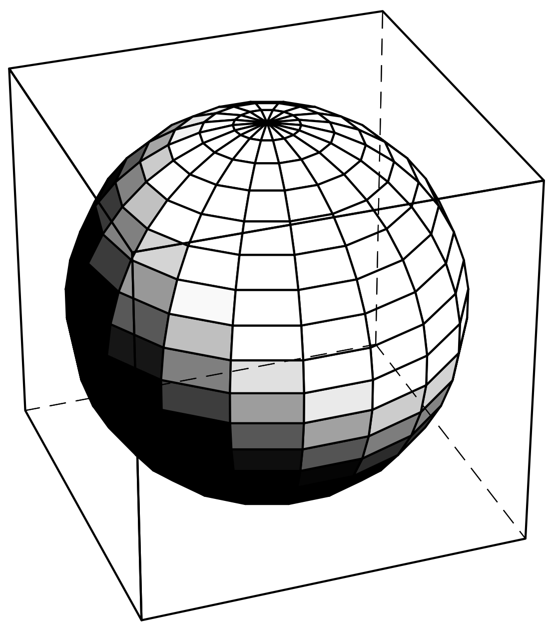

Figure 5.1: Computer graphic showing a sphere of unit diameter within a cube of unit side.

To determine if the efficiencies as theorised in section 3.3 were realised, the example of a spherically shaped room was used, as shown in figure 5.1. While the existence of such a shaped room seems far-fetched, the example illustrates some of the benefits of the data structure described even in somewhat unfavourable conditions. This configuration tests the performance of the new struct based method when a curved room shape is chosen. This curvature will necessarily result in partially empty block arrays being stored within structs at the boundary. The arrangement is also not particularly sparse. The volume of the sphere V_s is given by:

$$V_s = \frac{\pi d^3}{6} \tag{5.1}$$

meaning that that the fraction of points in the cube that are also inside the room is:

$$\frac{\pi}{6} \approx 0.5236... \tag{5.2}$$

This means that under the existing bounding box technique for modelling non-cuboid rooms, $47.6\ \%$ of points stored in the grid will be stored unnecessary. The amount by which this is reduced depends on ratio of dimensions between the sub-array within each struct (shown as Bx, By and Bz in listing 3.1) and the dimensions of the bounding box array, X, Y and Z. A simple experiment was done with the sphere example to measure this effect. The values of X, Y and Z are set as equal and can be thought of as the diameter of the sphere measured in grid points. A sub-array size of 8 by 8 by 8 was selected for ease of arithmetic. Spherical rooms were then generated with varying diameter sizes and converted to a semi-structured array of structs representation. The size of this array in memory was then compared with that yielded from the bounding box method. The result is shown in figure 5.2. Full results are included in appendix A.

When the size of the sub-array is equal to that of the bounding array, the semi-structured and structured approaches can be seen as equivalent. However, as shown in listing 3.1 there is srepeated ome overhead of storing the array indices of the neighbouring structs. This requires the storage of 6 integers for every struct, meaning that for a room of size 8 by 8 by 8 and with double precision floating point numbers, the semi-structured approach uses $1.002\,757$ times the memory of the structured approach, in line with the worst case scenario presented in section 3.1. This overhead is marginally higher when single precision numbers are used. The storage of these integers is still a very small overhead.

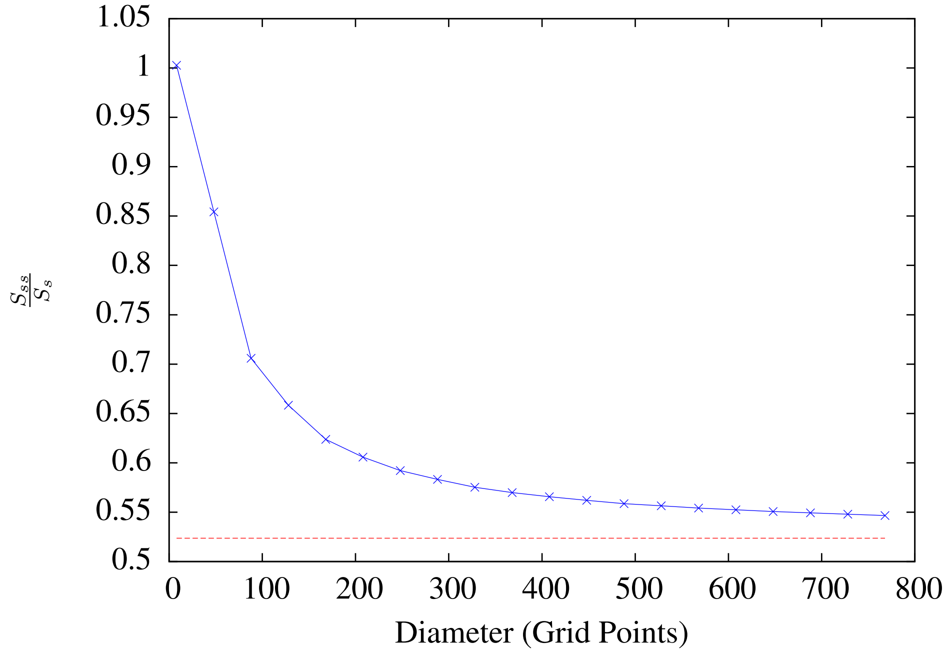

Figure 5.2: The fraction of memory used by the semi-structured approach with respect to the structured approach (double precision). The ideal fraction of $\frac{\pi}{6}$ is shown as a dashed line

The results shown in figure 5.2 show that as the diameter becomes larger, the storage efficiency of the semi-structured method increases. It is easy to see that should the diameter, d, become very large with respect to Bx, By and Bz, such that

$$\mathrm{Bx}, \mathrm{By}, \mathrm{Bz} \ll d \tag{5.3}$$

then the case is equivalent to the infinite sum of infinitesimals making up a spherical volume integral in Cartesian coordinates. This means that

$$\lim_{d \to \infty} \frac{S_{ss}}{S_s} = \frac{\pi}{6} \cdot 1.002757 \tag{5.4}$$

Practical gains come long before d approaches infinity. With a diameter of 768 points, the semi-structured approach means that $4014\,\mathrm{MiB}$ of global memory is required which falls within the 5GiB limit of the K20. The bounding box method for a room of this size would require $7344\,\mathrm{MiB}$ - meaning such a simulation could not be run on on a K20 graphics card.

5.1.2 Cube

The results from the cube shaped room experiment were very simple, as far as memory capacity goes. In all cases $\frac{S_{ss}}{S_s}$ was $1.002\,757$. This is because there is no empty space stored with the bounding box method. This represents the worst case performance for the semi-structured method. As such an increase in memory usage of around a quarter of one percent is a fairly tolerable result. Full results are included in appendix A.

5.1.3 L-shape or Cross????????

TO BE DONE

5.2 Stencil Performance

Chapter 6

Conclusions and Further Work

"It is not incumbent upon you to finish the task. Yet, you are not free to desist from it."

Tarphon

6.1 Future Work

Appendix A

Memory Capacity Experiment Results

The results are for double precision numbers.

Diam	Internal Blocks	Size semi-Structured (MiB)	Size - Structured (MiB)	Ratio
8	1	0.008324	0.008301	1.002757
48	184	1.531555	1.792969	0.854201
88	937	7.799278	11.048340	0.705923
128	2689	22.382347	34.000000	0.658304
168	5761	47.952660	76.873535	0.623786
208	10618	88.380722	145.894531	0.605785
248	17590	146.413345	247.288574	0.592075
288	27133	225.846130	387.281250	0.583158
328	39541	329.126221	572.098145	0.575297
368	55309	460.373840	807.964844	0.569794
408	74823	622.801941	1101.106934	0.565614
448	98430	819.298767	1457.750000	0.562030
488	126459	1052.602905	1884.119629	0.558671
528	159528	1327.858398	2386.441406	0.556418
568	197805	1646.463379	2970.940918	0.554189
608	241830	2012.912964	3643.843750	0.552415
648	291849	2429.254639	4411.375488	0.550680
688	348446	2900.349365	5279.761719	0.549333
728	411749	3427.262451	6255.228027	0.547904
768	482264	4014.206055	7344.000000	0.546597

Table A.1: Sphere memory capacity results

Diam	Internal Blocks	Size semi-Structured (MiB)	Size - Structured (MiB)	Ratio
8	1	0.008324	0.008301	1.002757
48	216	1.797913	1.792969	1.002757
88	1331	11.078804	11.048340	1.002757
128	4096	34.093750	34.000000	1.002757
168	9261	77.085503	76.873535	1.002757
208	17576	146.296814	145.894531	1.002757
248	29791	247.970428	247.288574	1.002757
288	46656	388.349121	387.281250	1.002757
328	68921	573.675598	572.098145	1.002757
368	97336	810.192688	807.964844	1.002757
408	132651	1104.143066	1101.106934	1.002757
448	175616	1461.769531	1457.750000	1.002757
488	226981	1889.314819	1884.119629	1.002757
528	287496	2393.021729	2386.441406	1.002757
568	357911	2979.132812	2970.940918	1.002757
608	438976	3653.891113	3643.843750	1.002757
648	531441	4423.539062	4411.375488	1.002757
688	636056	5294.319824	5279.761719	1.002757
728	753571	6272.476074	6255.228027	1.002757
768	884736	7364.250000	7344.000000	1.002757

Table A.2: Cube memory capacity results

Appendix B

NVIDIA K20 Device Details

Listing B.1: Output from deviceQuery program

```
CUDA Driver Version / Runtime Version          8.0 / 7.0
CUDA Capability Major/Minor version number:    3.5
 Total amount of global memory:                5062 MBytes
    (5307367424 bytes)
 (13) Multiprocessors, (192) CUDA Cores/MP:    2496 CUDA Cores
GPU Max Clock rate:                            706 MHz (0.71 GHz)
Memory Clock rate:                             2600 Mhz
Memory Bus Width:                              320-bit
L2 Cache Size:                                 1310720 bytes
Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536,
     65536), 3D=(4096, 4096, 4096)
Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048
    layers
Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384),
    2048 layers
 Total amount of constant memory:              65536 bytes
 Total amount of shared memory per block:      49152 bytes
 Total number of registers available per block: 65536
Warp size:                                     32
Maximum number of threads per multiprocessor:  2048
Maximum number of threads per block:           1024
Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
Max dimension size of a grid size    (x,y,z): (2147483647, 65535,
    65535)
Maximum memory pitch:                          2147483647 bytes
 Texture alignment:                            512 bytes
 Concurrent copy and kernel execution:         Yes with 2 copy engine
    (s)
Run time limit on kernels:                     No
 Integrated GPU sharing Host Memory:           No
 Support host page-locked memory mapping:      Yes
 Alignment requirement for Surfaces:           Yes
 Device has ECC support:                       Disabled
 Device supports Unified Addressing (UVA):     Yes
 Device PCI Domain ID / Bus ID / location ID:  0 / 2 / 0
```

Appendix C

Kernels

The following kernels where used.

Bibliography

[1] Webb, C.J. (2009) *Parallel computation techniques for virtual acoustics and physical modelling synthesis.* PhD thesis. University of Edinburgh. Available at: http://www2.ph.ed.ac.uk/~cwebb2/Site/Docs/CJWebb_thesis_FINAL.pdf (Accessed: 5 April 2017)

[2] Hamilton, B. (2016) *Finite Difference and Finite Volume Methods for Wave-based Modelling of Room Acoustics.* PhD thesis. University of Edinburgh. Available at: http://www2.ph.ed.ac.uk/~s1164563/thesis/hamilton2016phdthesis.pdf (Accessed: 5 April 2017)

[3] Richard Feynman, 1969, *Lectures in Physics, Volume 1,* Addison Publishing Company, Addison

[4] NVIDIA Corporation *What's the Difference Between a CPU and a GPU?* https://blogs.nvidia.com/blog/2009/12/16/whats-the-difference-between-a-cpu-and-a-gpu/ retrieved August 3, 2017.

[5] NVIDIA Corporation *NVIDIA Special Event: The GeForce GTX 1080 and Pascal (Part 4)* https://www.youtube.com/watch?v=0xnzwjPyx8A retrieved August 3, 2017.

[6] NVIDIA Corporation *CUDA C Programming Guide* http://docs.nvidia.com/cuda/cuda-c-programming-guide retrieved August 3, 2017.

[7] NVIDIA Corporation *NVIDIAs Next Generation CUDA Compute Architecture: Kepler GK110/210* http://images.nvidia.com/content/pdf/tesla/NVIDIA-Kepler-GK110-GK210-Architecture-Whitepaper.pdf retrieved August 3, 2017.

[8] P. Micikevicius, *3D finite difference computation on GPUs using CUDA*. Proceedings of 2nd Workshop on General Purpose Processing on Graphics Processing Units - GPGPU-2, New York, New York, USA, 2009, pp. 79-84, ACM Press.

[9] K. Hou, Y. Zhao, J. Huang, and L. Zhang, *Performance evaluation of the Three-Dimensional Finite-Difference Time-Domain(FDTD) method on Fermi architecture GPUs in Algorithms and Architectures for Parallel Processing* vol. 7016 of Lecture Notes in Computer Science, pp. 460-469. Springer Berlin / Heidelberg, 2011.

[10] J. Strikwerda, *Finite Difference Schemes and Partial Differential Equations*, Wadsworth and Brooks/Cole Advanced Books and Software, Pacific Grove, California, 1989.

[11] Harris, Frederic j. (Aug 2006). *Multirate Signal Processing for Communication Systems*. Upper Saddle River, NJ: Prentice Hall PTR. ISBN 0-13-146511-2

[12] Andreas Schäfer, Dietmar Fey, *High Performance Stencil Code Algorithms for GPGPUs*, Procedia Computer Science, Volume 4, 2011, Pages 2027-2036, ISSN 1877-0509, `http://dx.doi.org/10.1016/j.procs.2011.04.221.` (Accessed: 5 April 2017)

[13] D. W. Zingy , H. Lomax , and H. Jurgens , *High-accuracy finite-difference schemes for linear wave propagation*, SIAM J. Sci. Comput. 17, 328346, 1996.

[14] L. Savioja and U. P. Svensson, *Overview of geometrical room acoustic modeling techniques*, Journal of the Acoustical Society of America, vol. 138, no. 2, pp. 708730, 2015.

[15] Shapiro, L. G. and Stockman, G. C: *Computer Vision*, page 137, 150. Prentice Hall, 2001

[16] Tech Power Up *Graphics Card Market Up Sequentially in Q3, NVIDIA Gains as AMD Slips* `https://www.techpowerup.com/194979/graphics-card-market-up-sequentially-in-q3-nvidia/-gains-as-amd-slips` retrieved August 3, 2017.